

Mecanismos de Autenticação & Integridade

Funções de Dispersão (Hash)

Funções muito eficientes que mapeiam mensagens de qualquer tamanho para valores de tamanho fixo denominados *valores de hash*.

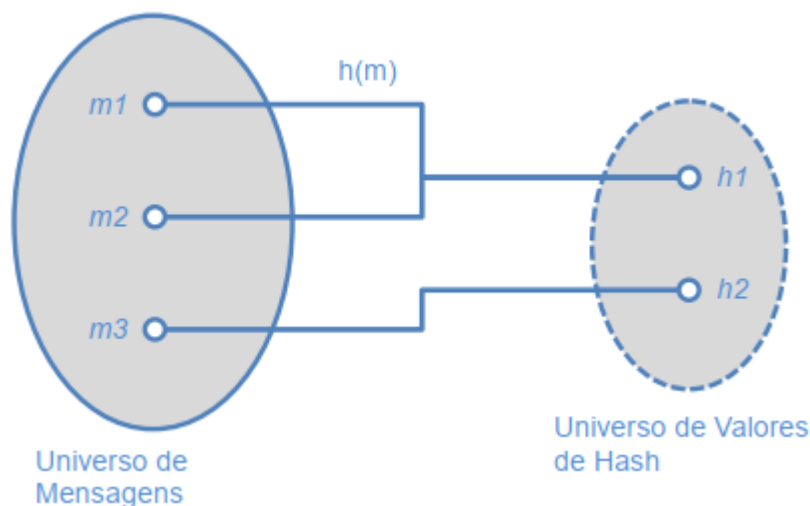
Funções de Hash Criptográficas

- Praticamente impossíveis de reverter.
- Número de valores de hash possíveis é finito → podem ocorrer **colisões**.
- Essenciais para garantir integridade da informação.

Propriedades para Segurança:

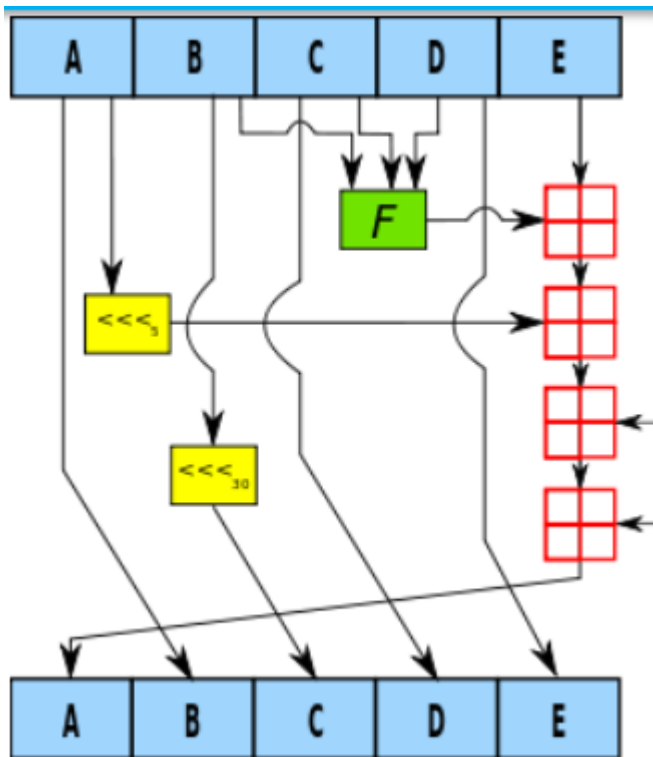
- Fácil cálculo do valor de hash.
- Devem ser **unidirecionais** (não se consegue obter a mensagem original).
- Devem ser **fortemente livres de colisões**.

Ilustração de Universos:

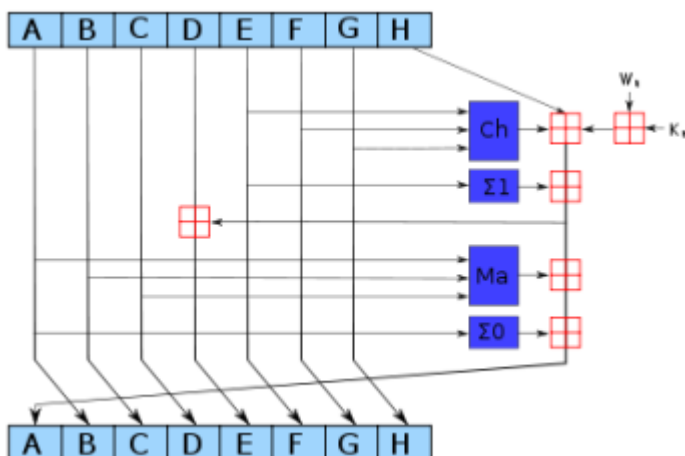


Algoritmos Populares:

- **MD5**: 128 bits – Inseguro desde 1996.
- **SHA-1**: 160 bits – Vulnerabilidades conhecidas.



- **SHA-2**: 224, 256, 384, 512 bits



- **SHA-3**: Baseado no algoritmo **Keccak**.

Assinatura Digital

Permite garantir:

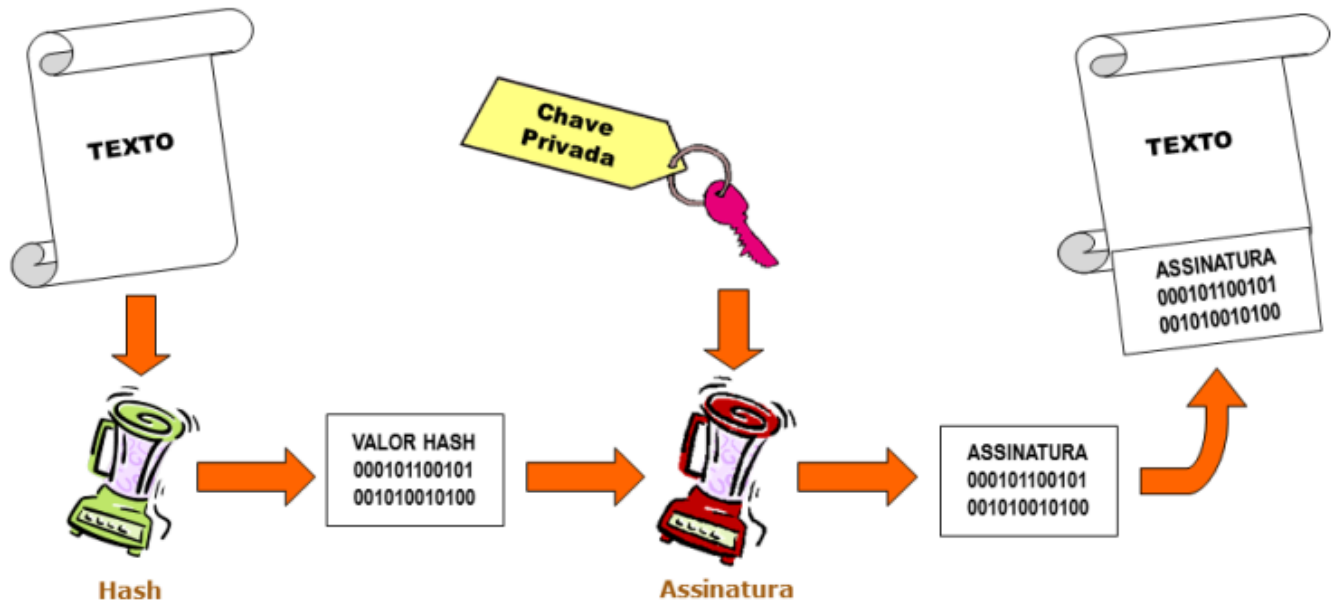
- Autenticidade
- Integridade
- Não-repúdio

Funcionamento (Exemplo com RSA)

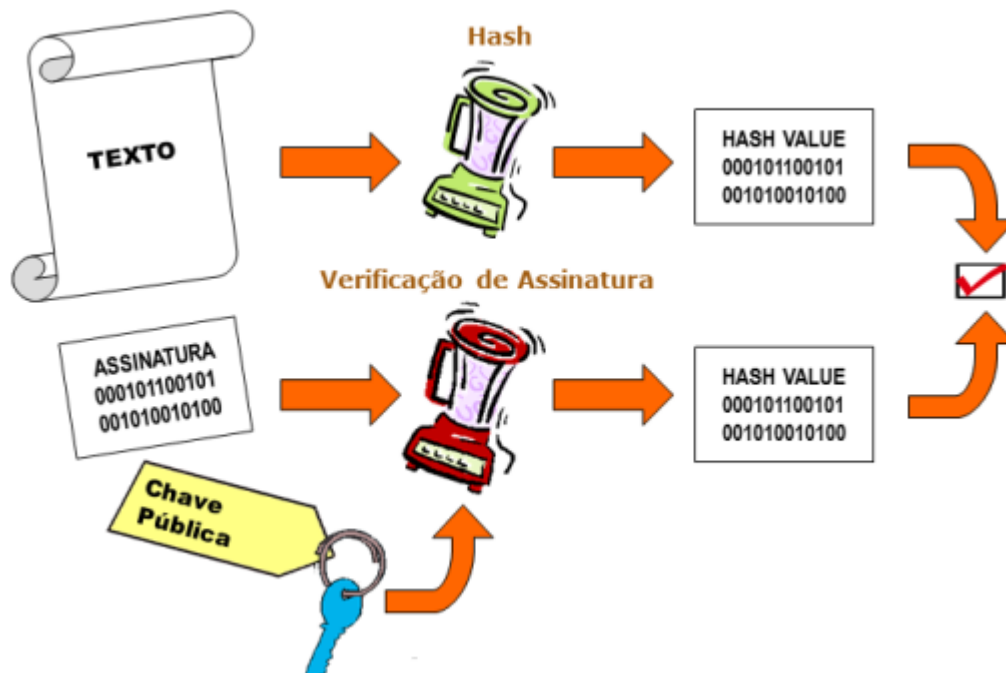
Math

- 1 Assinatura: $y = x^b \mod m$
- 2 Verificação: $x = y^a \mod m$

Assinatura:



Verificação:



Algoritmos:

- RSA
- El Gamal

- **DSA (Digital Signature Algorithm)**

El Gamal:

- Assinatura como par (s, w)
- Verificação: $w^s \cdot r^w \equiv b^x \pmod{p}$

DSA:

- Mais leve, mas verificação menos eficiente.
 - Parte do **DSS** com SHA-1 ou SHA-2.
-

MAC – Message Authentication Code

Autentica mensagens e garante integridade.

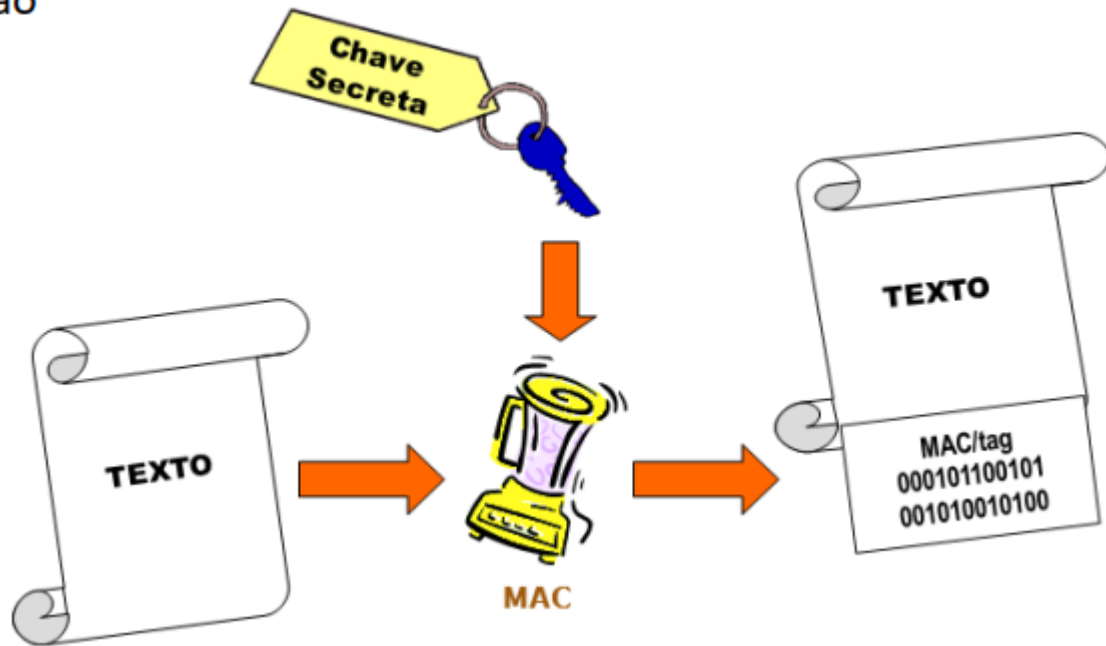
Requer chave **simétrica** compartilhada.

Vantagens:

- Mensagens de qualquer tamanho
- Combina integridade + autenticidade

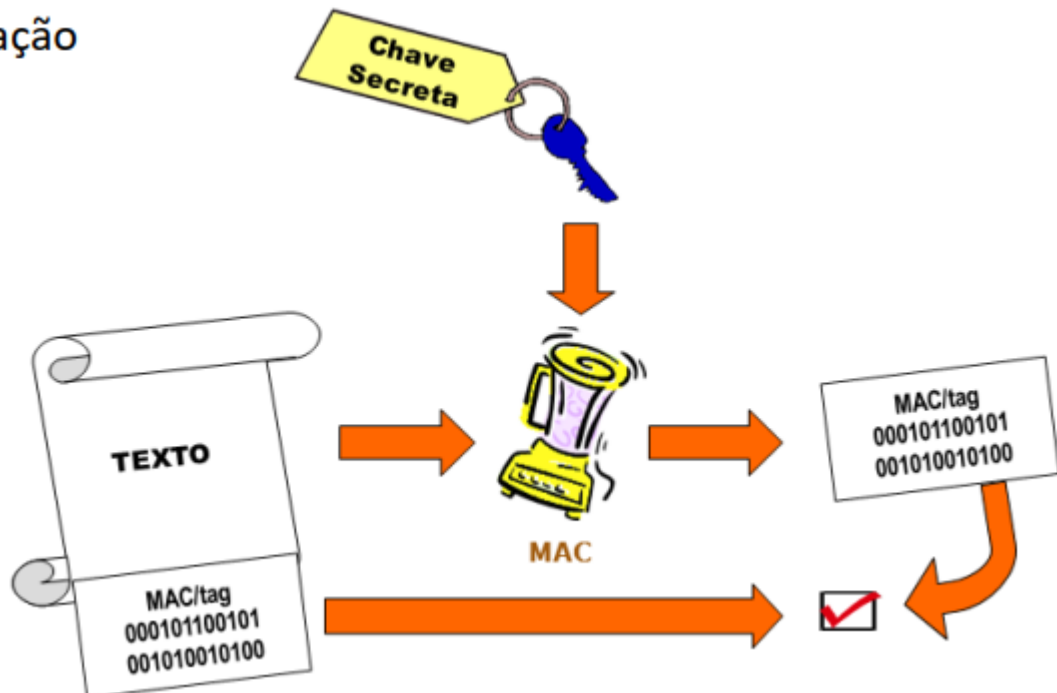
Ilustrações:

Criação:



Verificação:

ação



Propriedades:

- Chave simétrica
- Tamanho fixo
- Funciona com mensagens de tamanho variável
- Deteta alterações/manipulações

Diferenças:

- ≠ Hash: requer chave
- ≠ Assinatura: não garante não-repúdio

Algoritmos:

- **HMAC**: com hash (ex. SHA-2). RFC 2104.
 - **CBC-MAC**: baseado em cifras de bloco no modo CBC.
 - **CMAC**: versão segura para mensagens variáveis.
-

Protocolos de Acordo de Chave

Necessidade:

- Partilha segura de chave secreta.
- Proteção contra ataques *Man-in-the-Middle*.

Protocolos Comuns:

- **SSL (Secure Sockets Layer)**
 - **TLS (Transport Layer Security)**
-

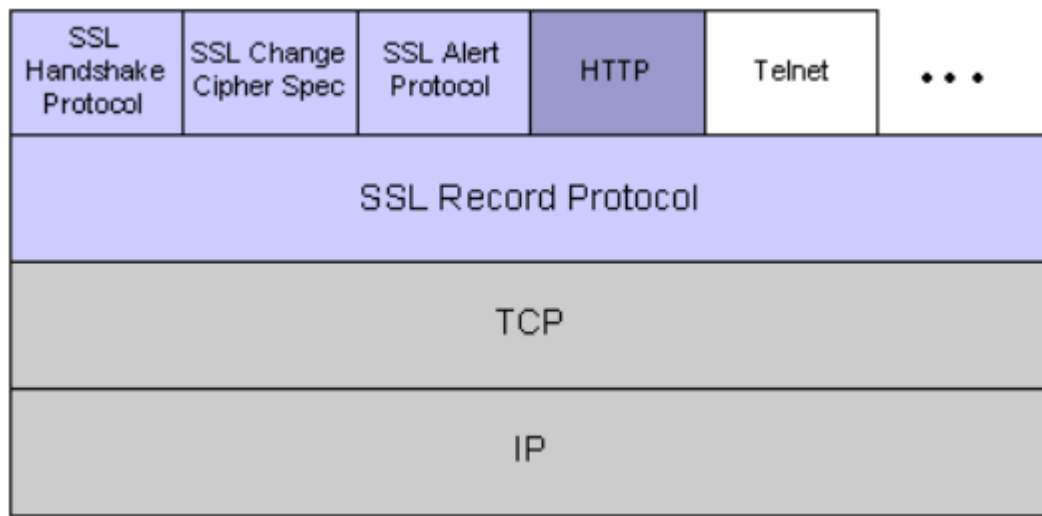
SSL (Secure Sockets Layer)

Publicado pela Netscape (1995).

Permite:

- Autenticação
- Integridade
- Confidencialidade

Arquitetura:



SSL Record Protocol:

- Fragmenta, comprime, cifra e autentica mensagens.

Estabelecimento de Sessão (6 Fases):

- 1. Pedido de Comunicação Segura**
 - Versão, algoritmos suportados, número aleatório.
- 2. Autenticação do Servidor**
 - Envio de certificado.
- 3. Autenticação do Cliente**
 - Cifra do pré-segredo com chave pública do servidor.
- 4. Cálculo da chave secreta**
- 5. MAC sobre comunicações anteriores**
- 6. Mudança para comunicação cifrada**
 - *SSL Change Cipher Spec*: ativa parâmetros acordados.
 - *SSL Alert Protocol*: sinaliza erros (encerra ligação).

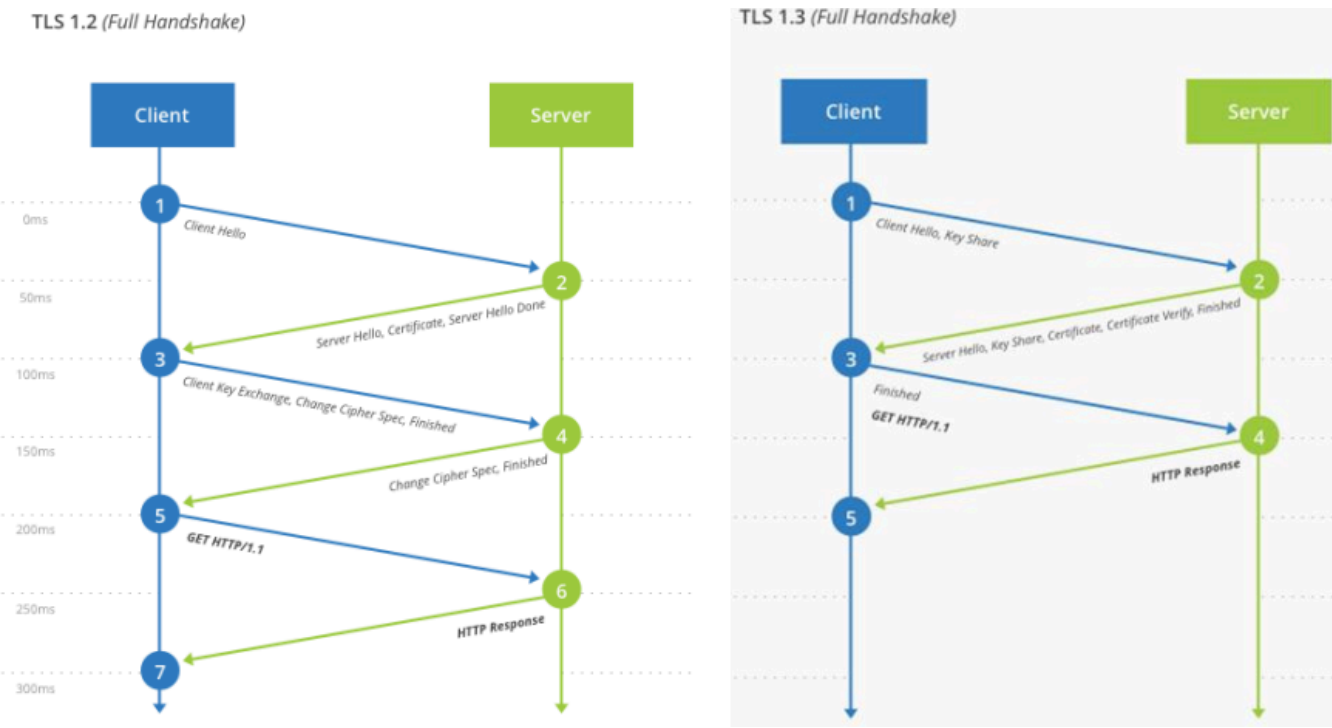
TLS (Transport Layer Security)

- Evolução do SSL (1999 → atual 1.3 RFC 8446).
- Compatível com HTTP/2
- Ligações mais rápidas (menos *round-trips*).

Diferenças principais (v1.3):

- Remoção de: MD5, SHA-1, RC4, DES, 3DES, AES-CBC.
- Algoritmos de MAC e derivação de chave reformulados.

Ilustração Comparativa:



Conclusão

Mecanismo	Garante Integridade	Garante Autenticidade	Requer Chave?	Garante Não-repúdio
Hash	Sim	Não	Não	Não
MAC	Sim	Sim	Sim (simétrica)	Não
Assinatura	Sim	Sim	Sim (assimétrica)	Sim